

Assessment-DBMS

Section A – Concept Application

1. How relational databases help maintain accuracy in expense records

Relational databases maintain accuracy by storing data in related tables using primary keys and foreign keys. In the Expense Tracker DB, users, categories, and expenses are stored separately but connected through relationships. This reduces duplicate data and ensures consistency. Every expense record must belong to a valid user and category.

2. Why constraints are important in personal finance data

Constraints help maintain data integrity and accuracy.
Examples:

- PRIMARY KEY prevents duplicate records
- FOREIGN KEY ensures valid relationships
- UNIQUE prevents duplicate emails
- NOT NULL avoids empty important fields

Without constraints, incorrect or duplicate expense records may occur.

3. How GROUP BY helps analyze spending patterns

GROUP BY organizes similar records into groups.
It helps calculate:

- Total expense category-wise
- Monthly spending
- User-wise expense analysis

This helps users understand spending habits easily.

4. Scenario where rollback is required during expense entry

Suppose a user accidentally enters an expense amount of 50000 instead of 500. ROLLBACK can undo the transaction before saving permanently. This restores the previous correct database state.

5. How views help users track monthly expenses efficiently

Views simplify complex queries and display only useful information. For example, a monthly expense view can show:

- User name

- Expense category
- Total monthly expenses

Views save time and improve readability.

6. Why use triggers for automatic category or balance updates

Triggers automatically perform actions after INSERT, UPDATE, or DELETE operations.

Examples:

- Auto-update balance after expense entry
- Auto-assign categories
- Maintain logs automatically

Triggers improve automation and accuracy.

Section B – SQL Hands-On

1. DDL Understanding

Why foreign keys are used

Foreign keys maintain relationships between tables.

In the expenses table, user_id must exist in the users table. This ensures every expense belongs to a valid user.

Issue if foreign keys are removed

Without foreign keys, orphaned records may occur.

Example:

An expense may exist for a deleted user, causing invalid data.

2. DML Operations

Insert 5 Users

```
INSERT INTO users VALUES
(1, 'Aryan', 'aryan@gmail.com', '2026-05-01'),
(2, 'Rahul', 'rahul@gmail.com', '2026-05-02'),
(3, 'Sneha', 'sneha@gmail.com', '2026-05-03'),
(4, 'Priya', 'priya@gmail.com', '2026-05-04'),
(5, 'Karan', 'karan@gmail.com', '2026-05-05');
```

Insert 3 Categories

```
INSERT INTO categories VALUES
(1, 'Food'),
(2, 'Rent'),
(3, 'Entertainment');
```

Insert 10 Expense Records

```
INSERT INTO expenses VALUES
(1, 1, 1, 250.00, '2026-05-10'),
(2, 1, 2, 5000.00, '2026-05-11'),
(3, 2, 1, 300.00, '2026-05-12'),
(4, 2, 3, 700.00, '2026-05-13'),
(5, 3, 2, 4500.00, '2026-05-14'),
(6, 3, 1, 200.00, '2026-05-15'),
(7, 4, 3, 1000.00, '2026-05-16'),
(8, 4, 1, 350.00, '2026-05-17'),
(9, 5, 2, 5500.00, '2026-05-18'),
(10, 5, 3, 1200.00, '2026-05-19');
```

Update Incorrect Expense

```
UPDATE expenses
SET amount = 800.00
WHERE expense_id = 4;
```

Delete One Expense

```
DELETE FROM expenses
WHERE amount < 250;
```

3. Data Retrieval

Display All Expenses with INNER JOIN

```
SELECT
e.expense_date,
e.amount,
u.name,
c.category_name
FROM expenses e
INNER JOIN users u
ON e.user_id = u.user_id
INNER JOIN categories c
ON e.category_id = c.category_id;
```

Total Expense Amount Per Category

```
SELECT
c.category_name,
SUM(e.amount) AS total_expense
FROM expenses e
INNER JOIN categories c
ON e.category_id = c.category_id
GROUP BY c.category_name;
```

Users Sorted by Total Spending

```
SELECT
u.name,
SUM(e.amount) AS total_spending
FROM users u
INNER JOIN expenses e
ON u.user_id = e.user_id
GROUP BY u.name
ORDER BY total_spending DESC;
```

4. Views

Create ActiveUsersView

```
CREATE VIEW ActiveUsersView AS
SELECT
u.name,
u.email
FROM users u
INNER JOIN expenses e
ON u.user_id = e.user_id
GROUP BY u.user_id, u.name, u.email
HAVING COUNT(e.expense_id) > 5;
```

Query the View

```
SELECT * FROM ActiveUsersView;
```

Section C – Mini Project

Expense Tracker DB

CRUD Operations

CREATE

```
INSERT INTO expenses
VALUES (11, 1, 1, 400.00, '2026-05-20');
```

READ

```
SELECT * FROM expenses;
```

UPDATE

```
UPDATE expenses
SET amount = 600.00
WHERE expense_id = 11;
```

DELETE

```
DELETE FROM expenses
WHERE expense_id = 11;
```

Stored Procedure for Monthly User Expense

DELIMITER //

```
CREATE PROCEDURE GetMonthlyExpense(
IN uid INT
)
BEGIN
```

```
SELECT
u.name,
MONTH(e.expense_date) AS month,
SUM(e.amount) AS total_expense
```

```
FROM users u
INNER JOIN expenses e
ON u.user_id = e.user_id
```

```
WHERE u.user_id = uid
```

```
GROUP BY MONTH(e.expense_date);
```

```
END //
```

DELIMITER ;

Execute Stored Procedure

```
CALL GetMonthlyExpense(1);
```

COMMIT Example

```
START TRANSACTION;
```

```
INSERT INTO expenses
VALUES (12, 2, 1, 300.00, '2026-05-21');
```

```
COMMIT;
```

ROLLBACK Example

```
START TRANSACTION;
```

```
INSERT INTO expenses
```

VALUES (13, 3, 2, 99999.00, '2026-05-22');

ROLLBACK;

Conclusion

This project demonstrates:

- SQL Queries
- CRUD Operations
- JOIN Operations
- GROUP BY
- Views
- Stored Procedures
- COMMIT and ROLLBACK

Output Examples

CREATE Output:

Record inserted successfully:

```
1 row inserted. INSERT INTO expenses VALUES (11, 1, 1, a 400.00, '2026-05-20' )
affected rows: 1
```

Query took 0.0021 sec.

READ Output:

Expense Table:

expense_id	user_id	category_id	amount	expense_date
1	1	1	320.00	2025-05-20
2	2	1	800.00	2025-05-11
3	2	1	503.00	2025-05-12
4	2	1	703.00	2025-05-13
5	2	2	602.00	2026-05-13
6	3	2	203.00	2025-05-14
7	4	1	100.00	2026-05-16
8	2	1	356.00	2026-05-17
9	4	1	356.00	2025-05-17
10	5	2	120.00	2025-05-10

UPDATE Output:

Record updated successfully:

```
1 row updated. UPDATE expenses SET amount = 600.00
WHERE expense_id = 11;
```

Rows matched: 1 Changed: 1 Warnings: 0

DELETE Output:

Record deleted successfully:

```
1 row deleted.
DELETE FROM expenses WHERE expense_id = 11;
```

Query took 0.0015 sec.

Stored Procedure Output:

Monthly Expense Result:

name	month	total_expense
Aryan	5	8650.00

COMMIT Output:

Transaction committed:

```
1 row inserted.
```

Query took 0.00012 sec. (Rows matched: 1 Changed: 1 Warnings: 0)

Transaction committed.

ROLLBACK Output:

Transaction rolled back:

```
1 row inserted.
```

Query took 0.0013 sec.

 Error: Operation failed.

There was an error while executing the query.
The transaction was rolled back.

11:41:20 ROLLBACK